

COP4934 Senior Design 1 Scrum & Meetings

Contents

1	Introduction	3
1.1	Sprint Planning	3
1.2	Sprint Review	3
1.3	Sprint Retrospective	3
2	Meeting Minutes	3
2.1	2-16-2026	3
2.2	2-23-2026	4
2.3	3-2-2026	5
2.4	3-9-2026	5
3	Senior Design 1	5
3.1	Sprint 0 : 2/1/2026 - 2/15/2026	5
3.1.1	Goal	5
3.1.2	Sprint Planning	5
3.1.3	Sprint Review	5
3.1.4	Sprint Retrospective	6
3.2	Sprint 1 : 2/15/2026 - 3/1/2026	6
3.2.1	Goal	6
3.2.2	Sprint Planning	6
3.2.3	Sprint Review	7
3.2.4	Sprint Retrospective	8
3.3	Sprint 2 : 3/1/2026 - 3/15/2026	8
3.3.1	Goal	8
3.4	Sprint Planning	8
3.4.1	Sprint Review	9
3.4.2	Sprint Retrospective	10
3.5	Sprint 3 : 3/15/2026 - 3/29/2026	10
3.5.1	Goal	10
3.5.2	Sprint Planning	10
3.5.3	Sprint Review	11
3.5.4	Sprint Retrospective	12
3.6	Sprint 4 : 4/12/2026 - 4/26/2026	13
3.6.1	Goal	13
3.6.2	Sprint Planning	13
3.6.3	Sprint Review	14
3.6.4	Sprint Retrospective	14
4	Senior Design 2	14
4.1	Sprint 5: 5/13/2026 - 5/27/2026	14
4.1.1	Goal	14
4.1.2	Sprint Planning	15
4.1.3	Sprint Review	15

4.1.4	Sprint Retrospective	15
4.2	Sprint 6 : 5/27/2026 - 6/10/2026	16
4.2.1	Goal	16
4.2.2	Sprint Planning	16
4.2.3	Sprint Review	16
4.2.4	Sprint Retrospective	17
4.3	Sprint 7 : 6/10/2026 - 6/24/2026	17
4.3.1	Goal	17
4.3.2	Sprint Planning	17
4.3.3	Sprint Review	18
4.3.4	Sprint Retrospective	18
4.4	Sprint 8 : 6/24/2026 - 7/8/2026	19
4.4.1	Goal	19
4.4.2	Sprint Planning	19
4.4.3	Sprint Review	19
4.4.4	Sprint Retrospective	20
4.5	Sprint 9 : 7/8/2026 - 7/22/2026	20
4.5.1	Goal	20
4.5.2	Sprint Planning	21
4.5.3	Sprint Review	21
4.5.4	Sprint Retrospective	22

1 Introduction

1.1 Sprint Planning

Sprint Planning is an agile event where the scrum team defines the work to be completed during the upcoming sprint. The team establishes a clear sprint goal, selects high-priority items from the product backlog, and breaks them into technical tasks to form the Sprint Backlog. This meeting aligns the team on what can be delivered by the end of the sprint and how the work will be accomplished.

- **Focus:** Planning the upcoming sprint and defining the sprint goal
- **Goal:** Decide what work will be done in the sprint and how it will be completed.
- **Participants:** Scrum Team (Developers, Product Owner, Scrum Master)
- **Outcome:** Sprint Goal, Sprint Backlog, and a shared understanding of the planned work

1.2 Sprint Review

Sprint Review is held at the end of a sprint to evaluate the completed product increment. The scrum team demonstrates finished work, gathers feedback from stakeholders, and discusses progress toward the overall product goals. The outcome of the review is an updated product backlog and a shared understanding of what was accomplished and what should be prioritized next.

- **Focus:** The Product Increment (what was built)
- **Goal:** Inspect the increment, get stakeholder feedback, and adapt the Product Backlog
- **Participants:** Scrum Team (Developers, PO, SM) + Stakeholders.
- **Outcome:** Updated Product Backlog, shared understanding of progress, feedback for future work

1.3 Sprint Retrospective

Sprint Retrospective is an internal scrum team meeting focused on improving the team's development process. The team reflects on what went well, what challenges were encountered, and how collaboration, tools, or workflows can be improved. The result is a set of actionable improvements that the team commits to applying in the next sprint.

- **Focus:** The Process (how the team works)
- **Goal:** Inspect the team's people, processes, and tools; identify improvements
- **Participants:** Scrum Team only (internal event)
- **Outcome:** Actionable improvements to the team's process/collaboration for the next sprint

2 Meeting Minutes

2.1 2-16-2026

Attendance: Everyone in the team attended the meeting.

During this meeting, we assigned the tasks for sprint 1. In the span of the next two weeks, each role will have their own assigned tasks dedicated for their roles, but we will work towards the overall objective of having a basic skeleton or backbone (low implementation) of our project set up and aim to have it dockerized and able to run on everyone's computer. Went through the Jira and added all the tasks and assigned them. Everyone will be in charge of updating their progress of each task on Jira. Below are the tasks that were assigned to each role:

- Cybersecurity/Honeypot Lead:
 - Choose honeypot method: decide on cowrite or custom SSH
 - Deploy first honeypot logging the user, pass, commands, and IP
 - Define threat boundaries: what attacker can do, what they cannot do, and what is logged
 - Export logs as JSON or CSV with one file per session
 - **Hard Deliverable:** A running SSH honeypot that logs real attacker sessions.
- Backend Systems Engineer:
 - Scaffold backend: Fast API, routes, models, services
 - Create API: POST, GET
 - Parse incoming honeypot logs: have framework to implement once backend is fully setup
 - Dockerize backend
 - **Hard Deliverable:** Backend service that accepts attack logs via HTTP.
- Database Engineer:
 - Design schema tables: sessions, events, credentials
 - Implement SQL init script
 - Connect backend to accept inserts and test queries
 - Write sample analytic queries
 - **Hard Deliverable:** Postgres with real tables storing attack data.
- Frontend Visualization Engineer:
 - Scaffold React App: layout, navigation
 - Create Views: sessions table and detail page
 - Connect backend: fetch and render data
 - Dockerize frontend
 - **Hard Deliverable:** Dashboard that displays real or mock sessions.
- ML Threat Analysis:
 - Define ML Problem: clustering or anomaly detection
 - Design features: session duration, command count, password entropy
 - Write pipeline: load data, compute features, plot distributions
 - **Hard Deliverable:** Jupyter notebook that processes attack data.

2.2 2-23-2026

Attendance: Everyone was present during this meeting.

Progress Update:

The team discussed individual progress on assigned roles and tasks. Overall, everyone has made significant progress and actively working towards completion on the current sprint's tasks. We are all consistently updating the Github and Jira as tasks are completed.

The backend is fully dockerized and we confirmed that it is running locally on everyone's machines/ The frontend is currently in progress and will be completed by the end of the week.

The team also reviewed requirements for the Design Proposal and Team Contract document that is due this weekend as well as what each of us needs to fill out.

Scheduling Update: There is a slight scheduling conflict of our meeting times since one of our members has a class during this time, so we agreed to moving our weekly meetings to Mondays at 1:30pm.

2.3 3-2-2026

Attendance: Everyone was present during this meeting.

Progress Update: Everything for sprint 1 was completed. First honeypot was deployed on Cowrie and it can send logs to the backend. We will research further on how to automate this in the future. This current sprint's tasks will be sent after the instructor check ins this week. We will update the Jira with the new tasks and assign them to their correct roles. We will also need to work on the Early status report that is due this weekend and provide MVP, Jira updates, sprint plannings, and Agile event info.

2.4 3-9-2026

Attendance: Everyone was present during this meeting.

Progress Update: In this meeting, we talked about the tasks for the next two sprints since they are connected. The main goal is to move Sentinel Grid to a real cloud honey net with a working dashboard and structured attack data. Zach sent out tasks for everyone for week 4 to 7 and we also updated the Jira board sprints and assigned tasks per role. Yama will start on the template for the Preliminary Design document so we can split up everyone's parts. We will work parallel to efficiently get tasks done. Next week is spring break, so we will just have a basic progress update on implementation and documentation.

3 Senior Design 1

3.1 Sprint 0 : 2/1/2026 - 2/15/2026

3.1.1 Goal

Our goal for Sprint 0 is to set the foundation of our project and to establish a clear semester plan.

3.1.2 Sprint Planning

The focus of Sprint 0 is to plan the foundation required to establish a clear semester plan. Our team talked about our skills, preferred programming languages, experience, and what we would each like to do in order to figure out everyone's roles and responsibilities. This will help us figure out what each member needs to develop and learn this semester and clear up any questions before we start implementation.

- What can we deliver by the end of this sprint?
By the end of this sprint, we will deliver a draft of our semester plan that clearly defines team roles, responsibilities, learning gaps, and requirements of the project.
- How will we build it?
We will have a meeting to collaboratively develop the semester plan including planning weekly meetings. Responsibilities will be assigned based on individual strengths and learning goals, and the plan will be documented and reviewed to ensure alignment with project objectives.
- What must be done vs nice:
Must be done: defining team roles, assigning responsibilities, identifying required skills and knowledge
Nice to have: detailed stretch goals and general requirements

3.1.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
Weekly meeting times were agreed upon to take place online every Mondays at 12pm and in person meetings will take place as needed. We defined team roles, assigned responsibilities, each member had an idea of required skills and tools they needed to learn.

- What worked well:
Communication about everyone's background and skills helped the team assign roles effectively. Collaboration during planning ensured that everyone felt heard and aligned with the project goals and roles.
- Feedback received:
Its important to have certain milestones and deliverables for each sprint. It was also suggested that documentation should be kept updated throughout the semester for each member so we can keep track of what everyone is working on.
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
Added tasks for researching required technologies, setting up the GitHub, and creating documentation repository.

3.1.4 Sprint Retrospective

The team discussed:

- What went well during the sprint:
The team established a clear direction for the semester and ensured everyone understood their responsibilities. Meetings were productive and stayed focused on deliverables and what each person had to do next.
- What challenges or obstacles were encountered:
Aligning everyone's schedules for the weekly meetings and there was a learning curve when it came to learning how to use and understand Jira
- What could be improved in future sprints: The team should also break large goals into smaller, more manageable tasks to improve tracking and accountability. Each member is able to create their own sub tasks and other things they are working on and add them to the Jira and update them as they complete them
- Action items to implement in the next sprint:
Set up the development environment, work on project requirements, assign tasks for the next spring

3.2 Sprint 1 : 2/15/2026 - 3/1/2026

3.2.1 Goal

Our goal for Sprint 1 is to complete the design proposal and team contract with the focus on developing the skills and knowledge required for each team role while working on each respective assigned tasks.

3.2.2 Sprint Planning

During the 2/16/2026 meeting, all team members were present and Sprint 1 tasks were assigned. Each team member received role specific responsibilities to work on building a basic skeleton and low implementation of the project. The team aims to have a functional backbone of the system set up, dockerized, and capable of running on all team members' machines by the end of the sprint. All tasks were added to Jira, and each member is responsible for updating their task progress throughout the sprint.

- What can we deliver by the end of this sprint?
 - A running SSH honeypot that logs attacker sessions (credentials, commands, IP)
 - A dockerized backend service that accepts attack logs
 - A Postgres database with implemented schema tables (sessions, events, credentials) storing attack data
 - A dockerized React frontend that displays real or mock session data

- A Jupyter notebook that processes attack data for clustering or anomaly detection
- A basic, integrated project skeleton that runs locally on all team members' machines
- How will we build it?
Each role will focus on completing its assigned deliverables
- What must be done vs nice:
Must be done:
 - Dockerize backend and frontend
 - Ensure the system skeleton runs locally for everyone
 - Complete the design proposal and team contract.

Nice to have:

- Complete all hard deliverables for each role
- Initial integration testing between all components
- Basic analytics visualizations beyond required views

3.2.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Design proposal and team contract was completed
 - Backend and data base set up and dockerized
 - Front end scaffold completed
 - ML notebooks completed for data analysis and data preprocessing pipeline
 - Docker for backend verified to run successfully across everyone's environments locally
 - Deployed first honeypot on cowrie that can be compromised via ssh root login with no proper authentication
- What worked well:
 - Separate github branches and roles allowed for parallel development across backend, frontend, database, and ml
 - Weekly meetings for updates
 - Regular communication and working through issues together on Discord
 - Jira task tracking for progress and accountability
- Feedback received:
 - Clarity and good documentation needed for other members to know how to run certain parts and ensure reproducibility
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
 - Built off of this sprint to add the next steps and tasks for the next sprint
 - Idea of exploring more possibilities of supervised learning for classifications

3.2.4 Sprint Retrospective

The team discussed:

- What went well during the sprint:
 - Every member successfully worked on their roles and tasks while learning the technologies they will be using
 - We have established a project skeleton earlier than expected
 - Communication remained consistent
- What challenges or obstacles were encountered:
 - Some of us had to learn how to use docker for the first time
 - Limited availability of realistic labeled attack data cause restriction in experimenting with supervised classification- will need to find another way
- What could be improved in future sprints:
 - Aim to complete assignments the day before so we all can review the day of submissions
 - Improved documentation updates on Github for readmes for different parts of the project structure
- Action items to implement in the next sprint:
 - Look into supervised learning for classification
 - Look into how to connect all the different parts together into a functioning structure (connect back end to ml, to front end, to data, etc.)
 - Not specifically for next sprint but will need to look into automatic sending logs from honeypot to backend in the future

3.3 Sprint 2 : 3/1/2026 - 3/15/2026

3.3.1 Goal

Establish the core cloud infrastructure for SentinelGrid so we can move beyond a local Docker only setup. By the end of Sprint 2, the team aims to have the backend prepared for cloud deployment, the database configured for real log storage, at least one honeypot connected to the backend log pipeline, and the frontend prepared for integration with live API data. Also changed meeting times to Mondays at 1:30

3.4 Sprint Planning

Sprint 2 focuses on turning SentinelGrid from a local prototype into the first version of a cloud-connected cybersecurity monitoring platform. The priority is to deploy or prepare the backend and database in the cloud, confirm that honeypot logs can be accepted and stored through the API, and ensure the frontend can begin consuming real backend data. Since this is the first infrastructure sprint of the new phase, the emphasis is on reliability, connectivity, and end-to-end data flow rather than advanced analytics or polished UI features.

- What can we deliver by the end of this sprint?
 - A cloud-hosted backend deployment or a fully prepared backend deployment environment on a VPS
 - A PostgreSQL database instance configured for SentinelGrid and connected
 - A verified log ingestion pipeline where honeypot events can be sent to the backend and stored in the database
 - At least one deployed honeypot node producing structured logs

- Backend API endpoints tested for basic functionality such as log submission and session retrieval
- A frontend/dashboard environment connected to backend APIs or prepared for deployment and live integration
- Updated documentation for deployment, infrastructure setup, and ingestion workflow
- Preprocessing data pipeline for ML and feature extraction
- How will we build it? We will work and build each part of the structures in parallel on our own branches on Github. Those relying on other parts needing to be setup first will conduct research on what they need to do/start with a skeleton baseline
- What must be done vs nice:
 - Must be done:**
 - Set up cloud infrastructure for backend deployment
 - Deploy or finalize deployment-ready FastAPI backend
 - Set up PostgreSQL database and confirm schema creation
 - Connect backend to database successfully
 - Insert incoming logs into database correctly
 - Deploy at least one
 - Fix frontend integration with backend APIs
 - Document setup steps and team workflow
 - ML pipelines for data and features

Nice to have:

- Add automatic log forwarding scripts instead of manual testing
- Add input validation and stronger API error handling
- Start preprocessing modularization for ML pipeline integration
- Prepare early visualizations for attack sessions if enough real logs are available

3.4.1 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Set up the cloud VPS environment for deploying SentinelGrid services
 - Successfully deployed the FastAPI backend and connected it to the PostgreSQL database
 - Implemented backend API endpoints for receiving and storing honeypot logs
 - Connected the Cowrie honeypot to the backend and verified log ingestion
 - Continued development of the React dashboard and integrated live API data
 - Expanded the data preprocessing pipeline and began extracting session-level features for future analysis
 - Updated project documentation for deployment, backend setup, and database configuration
- What worked well:
 - Team members were able to work in parallel on separate components with minimal merge conflicts
 - Cloud deployment allowed everyone to test against the same backend environment
 - Regular communication through Discord and weekly meetings helped resolve integration issues quickly

- Feedback received:
 - Continue improving documentation for deployment and environment setup
 - Ensure all API endpoints are consistently tested before frontend integration
 - Begin thinking about how collected attack data will be processed and categorized later down the road
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
 - Added tasks for automatic log forwarding from honeypots
 - Added backend analytics endpoints for session summaries and attacker statistics

3.4.2 Sprint Retrospective

The team discussed:

- What went well during the sprint:
 - The backend, database, and infrastructure were successfully connected into a functioning system
 - Team collaboration remained consistent and everyone completed their assigned sprint tasks
 - Early integration testing helped identify any issues between project components
- What challenges or obstacles were encountered:
 - Integrating multiple independently developed components required additional debugging
 - Some frontend features had to wait until backend APIs were finalized
- What could be improved in future sprints:
 - Perform integration testing more frequently throughout development
 - Define API contracts earlier to reduce frontend and backend dependency issues

3.5 Sprint 3 : 3/15/2026 - 3/29/2026

3.5.1 Goal

Expand SentinelGrid from a cloud-connected prototype into a more complete live cybersecurity monitoring platform. By the end of Sprint 3, the team aims to stabilize the deployed infrastructure, support multiple honeypot nodes, improve structured log storage and analytics queries, complete the main dashboard views, and prepare the ML pipeline for anomaly detection, clustering, and backend integration.

3.5.2 Sprint Planning

Sprint 3 focuses on system completion, analytics, and stability. After establishing the cloud deployment and initial log-ingestion pipeline in Sprint 2, this sprint will extend SentinelGrid into a more functional honeynet platform with multiple honeypots, improved database performance, new backend analytics endpoints, and a more complete React dashboard. In parallel, the ML/data pipeline will move from preprocessing and baseline feature extraction toward clustering and anomaly scoring so that attack sessions can later be surfaced as higher-level patterns rather than raw logs alone. ML will also explore the possibility of labeling data for supervised classification

- What can we deliver by the end of this sprint?
 - Two deployed honeypot nodes, including one SSH honeypot and one HTTP honeypot
 - Automatic log forwarding from honeypots into the backend API
 - A stable PostgreSQL database storing structured live attack data

- Improved database schema and indexes for faster log queries
- Analytics endpoints for top attacker IPs, attack rates, commands, and session summaries
- A working React dashboard with pages such as Live Attack Feed, Session Explorer, and Attack Analytics
- Visualizations for attack frequency, common commands, and top sources of activity
- Initial clustering and anomaly detection outputs from honeypot sessions and external datasets
- Documentation for infrastructure, API usage, database maintenance, and ML integration
- How will we build it? We will continue to work in parallel on our own branches on Github. Those relying on other parts needing to be setup first will conduct research on what they need to do/start with a skeleton baseline

- What must be done vs nice:

Must be done:

- Deploy and maintain both honeypots
- Ensure automatic log forwarding works
- Improve raw log schema and add indexes on important fields such as timestamp, IP, and session ID
- Implement backend analytics endpoints
- Complete core dashboard pages for live logs, sessions, and analytics
- Add charts for attack trends and attacker activity
- Run clustering or anomaly detection pipeline on collected session data
- Define backend ready output format for anomaly or behavior analysis results

Nice to have:

- Add automatic database backups and export scripts
- Create fake attacker interactions such as fake admin logins or fake filesystem responses
- Add auto refresh and improved filtering in the dashboard
- Add richer session drill-down features in the UI
- Tune anomaly scoring for near real time detection
- Integrate early anomaly scores directly into dashboard views
- Improve UI responsiveness and polish for demo readiness
- Possible labeling subset for ML supervised classification

3.5.3 Sprint Review

The team reviewed the following:

- **What was completed during the sprint:**

- Successfully deployed Cowrie (SSH) honeypots
- Implemented automatic log forwarding from each honeypot to the FastAPI backend using REST API endpoints
- Improved the PostgreSQL database schema by separating logs into structured tables and adding indexes on timestamp, source IP, session ID, and event type to improve query performance
- Expanded the React frontend with the Live Attack Feed, Session Explorer, and Attack Analytics pages

- Built the initial machine learning pipeline for preprocessing session data, extracting behavioral features, performing K-Means clustering, and detecting anomalous sessions using Isolation Forest
- Established a standardized backend output format for machine learning results to simplify future dashboard integration
- **What worked well:**
 - Parallel development allowed frontend, backend, infrastructure, and machine learning tasks to progress simultaneously with minimal blocking
 - The modular FastAPI architecture made it straightforward to add new analytics endpoints without affecting existing functionality
 - The machine learning feature engineering pipeline successfully generated meaningful behavioral features that separated different attacker activities into distinct clusters with the use of kmeans
- **Feedback received:**
 - Improve dashboard filtering options to better navigate large numbers of attack sessions
 - Continue improving the visual design and responsiveness of dashboard components
 - Increase automation for infrastructure deployment and database maintenance where possible
- **Updates made to the product backlog (new items, changes in scope, or priority adjustments):**
 - Added support for additional honeypot services such as FTP, Redis, and MySQL
 - Prioritize backend performance optimization for larger log volumes
 - Plan implementation of heuristic attack labeling and decision rules for adaptive honeypot deployment.

3.5.4 Sprint Retrospective

The team discussed:

- **What went well during the sprint:**
 - Infrastructure became considerably more stable, allowing continuous collection of live attack data throughout the sprint
 - Team members remained productive by working independently on well-defined components while communicating regularly through GitHub and sprint meetings
 - Machine learning experiments successfully demonstrated that behavioral clustering and anomaly detection could identify different categories of attacker activity
 - The dashboard evolved into a functional monitoring platform suitable for demonstrating live cyber attacks and analytics
- **What challenges or obstacles were encountered:**
 - Some dashboard queries became slower as the amount of collected data increased before database indexes were optimized
 - Live cloud deployments required additional monitoring and debugging to ensure reliable log forwarding
- **What could be improved in future sprints:**
 - Increase automated testing for backend APIs and frontend components before deployment
 - Improve deployment automation

- Expand feature engineering using additional temporal and behavioral metrics to improve machine learning performance.
- Continue refining the dashboard interface to make navigation more intuitive for non technical users

- **Action items to implement in the next sprint:**

- Develop heuristic attacker labeling and decision rules for automated attack classification
- Expand support for additional honeypot protocols and larger scale deployments

3.6 Sprint 4 : 4/12/2026 - 4/26/2026

3.6.1 Goal

Integrate all major SentinelGrid components and prepare for the Preliminary Design Review. By the end of Sprint 4, the team aims to demonstrate a fully functioning pipeline from honeypot attack collection through backend processing, database storage, planned machine learning analysis + pipeline, with dashboard visualization.

3.6.2 Sprint Planning

Sprint 4 focuses on integration, testing, and documentation. After completing the core infrastructure during the previous sprint, this sprint will ensure all currently implemented project components work reliably. The team will also prepare documentation and required information and materials for the Preliminary Design Document. We will prepare for Senior Design 2, leaving us with a good place to start once the new semester starts.

- What can we deliver by the end of this sprint?
 - Stable dashboard displaying live attack information
 - Backend integration for ML analysis results
 - Preliminary Design Documentation
 - Additional testing and bug fixes across the system
- How will we build it?

Continue parallel development while prioritizing component integration, testing, documentation, and bug fixes. Frequent integration meetings will be held to quickly resolve interface and deployment issues. The Preliminary Design Document will be split up between roles and each member will have their designated sections they will need to complete
- What must be done vs nice:

Must be done:

 - Complete dashboard integration
 - Perform integration and deployment testing
 - Complete Preliminary Design documentation

Nice to have:

- Improve dashboard responsiveness
- Additional analytics visualizations
- Better deployment automation
- Additional API documentation

3.6.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Integrated backend APIs with frontend dashboard
 - Connected ML analysis pipeline with backend processing
 - Completed Preliminary Design documentation
 - Fixed multiple integration bugs discovered during testing
- What worked well:
 - Integration testing greatly improved overall system reliability
 - Clear communication minimized merge conflicts
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
 - Prioritize multiple honeypot deployment
 - Expand adaptive honeypot research
 - Improve dashboard usability

3.6.4 Sprint Retrospective

The team discussed:

- What challenges or obstacles were encountered:
 - Integration testing uncovered unexpected API inconsistencies
 - Coordinating updates across multiple components required additional effort
- What could be improved in future sprints:
 - Begin integration testing earlier
 - Improve API documentation
 - Expand testing
- Action items to implement in the next sprint:
 - Begin planning the implementing adaptive honeynet features
 - Expand supported honeypot services

4 Senior Design 2

4.1 Sprint 5: 5/13/2026 - 5/27/2026

4.1.1 Goal

Begin Senior Design II by completing checkin and prepare for our Critical Design Review (CDR). The focus will be refining the overall architecture, improving deployment reliability, and planning the remaining features required for the final demonstration.

4.1.2 Sprint Planning

- What can we deliver by the end of this sprint?
 - CDR preparation
 - Improved deployment workflow
 - Updated project backlog and implementation schedule
- How will we build it? Continue improving existing components while incorporating review and peer feedback to prepare for the Critical Design Review.

Must be done:

- Prepare for the CDR
- Practice Presentation
- Improve documentation and plan the rest of SD2

Nice to have:

- Improve dashboard appearance
- Additional testing automation
- begin deployment of additional honeypots

4.1.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Started the preparation of the Critical Design Review presentation.
 - Refined the deployment process to simplify backend and database setup.
 - Reviewed the remaining product backlog and prioritized features for the remainder of the semester.
- What worked well:
 - Team members collaborated effectively on presentation preparation.
 - Existing project components were stable enough to demonstrate the current system.
 - Documentation improvements made it easier to explain the system architecture and development progress.
- Feedback received:
 - Checkin did not go as planned, but we will use the feedback to get back on track

4.1.4 Sprint Retrospective

The team discussed:

- What challenges or obstacles were encountered:
 - Balancing presentation preparation with continued development
 - Identifying deployment issues across different development environments
- Action items to implement in the next sprint:
 - Begin implementing the remaining high priority backlog items
 - Expand honeypot deployment and integration testing

4.2 Sprint 6 : 5/27/2026 - 6/10/2026

4.2.1 Goal

Expand SentinelGrid into a complete adaptive honeynet by deploying multiple honeypot services and containerizing each deployment with Docker for scalable management.

4.2.2 Sprint Planning

- What can we deliver by the end of this sprint?
 - SSH, HTTP, FTP, Redis, MySQL and SMTP honeypots
 - Docker deployment for every honeypot
 - Centralized logging
 - Unified backend support
- How will we build it? Develop and deploy each honeypot independently before integrating them into the shared backend infrastructure.
- Must be done:
 - Deploy all supported honeypots
 - Dockerize deployments
 - Normalize logs
 - Backend compatibility

Nice to have:

- Automatic deployment scripts
- Health monitoring
- Container scaling

4.2.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Successfully deployed SSH (Cowrie), HTTP, FTP and will work on others like Redis, MySQL, and SMTP honeypot services in the next sprint
 - Containerized each honeypot using Docker for consistent deployment and easier management
 - Configured centralized log collection to forward events from all honeypots to the backend.
 - Extended the backend to support multiple honeypot types with a unified logging interface
- What worked well:
 - Docker significantly simplified deployment and testing across different honeypot services
 - The modular backend architecture allowed new honeypots to be integrated with minimal changes
 - Independent development of each honeypot allowed easier testing of each type
- Feedback received:
 - Add monitoring for container health and service availability
 - Ensure consistent metadata is captured across all honeypot logs for future analytics

- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
 - Added automated Docker deployment and configuration scripts
 - Add container health monitoring and automatic restart capabilities
 - Prioritized adaptive honeypot redistribution using the machine learning decision engine
 - Planned dashboard enhancements to visualize activity across multiple honeypot types

4.2.4 Sprint Retrospective

The team discussed:

- What challenges or obstacles were encountered:
 - Different honeypots produced varying log formats, requiring additional normalization logic
 - Some Docker networking and port configuration issues delayed deployment
 - Maintaining consistent configurations across multiple containers required additional coordination
 - Took a while to get the honeypots up and running
- What could be improved in future sprints:
 - Automate deployment and configuration using scripts
 - Improve monitoring and alerting for container failures
 - Create standardized deployment templates to simplify adding new honeypot services
- Action items to implement in the next sprint:
 - Integrate the machine learning pipeline with the newly deployed honeypots
 - Begin developing adaptive honeypot redistribution based on detected attack behavior
 - Enhance the frontend to display activity from all deployed honeypot services
 - Finish the implementation of the variety of honeypots

4.3 Sprint 7 : 6/10/2026 - 6/24/2026

4.3.1 Goal

Develop behavioral analysis engine by generating heuristic attacker profiles and implementing decision rules that recommend optimal honeypot deployments based on recent attack behavior.

4.3.2 Sprint Planning

- What can we deliver by the end of this sprint?
 - Behavioral feature engineering
 - Heuristic attacker labeling
 - Decision engine
 - Recommendation API
 - Dashboard visualization
- How will we build it?

Continue extending the machine learning pipeline by combining clustering, feature engineering, anomaly detection, heuristic rules, and backend integration.

- What must be done vs nice:

Must be done:

- Behavioral analysis
- Decision rules
- Backend APIs
- Frontend visualization

Nice to have:

- Automatic deployment recommendations
- Confidence scores
- Recommendation history

4.3.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Developed a behavioral analysis pipeline that extracts attacker behavior metrics from normalized session data
 - Implemented heuristic labeling to classify attacker profiles based on clustering results, anomaly detection, and behavioral features
 - Built a decision engine that recommends appropriate honeypot deployments using predefined decision rules
 - Integrated recommendation APIs into the backend for real time access results
- What worked well:
 - Combining feature engineering with clustering and anomaly detection produced meaningful attacker profiles
 - Backend APIs integrated smoothly with the frontend, enabling near real time visualization of recommendations on a set interval
- Feedback received:
 - Include confidence scores to indicate the reliability of heuristic classifications
 - Consider storing historical recommendations to evaluate changes in attacker behavior over time
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
 - Add recommendation history and trend analysis
 - Added confidence scoring for heuristic attacker profiles
 - Prioritized automatic honeypot deployment based on decision engine recommendations

4.3.4 Sprint Retrospective

The team discussed:

- What challenges or obstacles were encountered:
 - Fine tuning heuristic thresholds required multiple iterations to balance accuracy and generalization.
 - Integrating outputs from clustering and anomaly detection into a unified recommendation

- Processing larger log datasets increased execution time
- What could be improved in future sprints:
 - Optimize feature engineering and model execution for improved performance
 - Expand behavioral features to support additional honeypot services and attack types
- Action items to implement in the next sprint:
 - Implement automatic honeypot redistribution the decision engine

4.4 Sprint 8 : 6/24/2026 - 7/8/2026

4.4.1 Goal

Integrate the adaptive honeypot recommendation engine into the dashboard while providing users with interactive controls for manually managing honeypot deployments and visualizing live attacks.

4.4.2 Sprint Planning

- What can we deliver by the end of this sprint?
 - Adaptive dashboard
 - Live world map
 - Honeypot distribution controls
 - Deployment recommendations
- How will we build it?

Connect backend recommendation APIs directly to the React dashboard while expanding visualization capabilities
- What must be done vs nice:

Must be done:

 - Recommendation dashboard
 - World attack map
 - Manual deployment controls
 - Live updates

Nice to have:

 - Geographic statistics
 - Notification settings
 - Moving around map and zoom

4.4.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Integrated the adaptive honeypot recommendation engine into the dashboard
 - Developed a live world map displaying attacker locations, ips, event types, and active attack events
 - Implemented manual honeypot deployment controls, allowing users redistribute honeypot services

- Connected backend recommendation APIs to the frontend, providing live deployment recommendations based on recent attacker behavior
- Added automatic dashboard updates to display current honeypot distribution without requiring manual refreshes.
- What worked well:
 - Backend APIs integrated smoothly with the dashboard, enabling real time communication between the recommendation engine and frontend
 - Interactive visualizations made it easier to understand attack activity and deployment decisions.
 - Manual deployment controls provided users with flexibility while maintaining automated recommendations.
- Feedback received:
 - Improve geographic visualizations by providing additional country and regional statistics.
 - Allow users to filter attacks by honeypot type, attacker profile, and time range.
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
 - Added geographic analytics and attack trend reporting.
 - Planned enhanced filtering and search capabilities within the dashboard

4.4.4 Sprint Retrospective

The team discussed:

- What challenges or obstacles were encountered:
 - Synchronizing backend recommendations with manual deployment actions required careful state management.
 - Displaying geographic attack information accurately depended on external IP geolocation services.
 - Testing overall deployment
- What could be improved in future sprints:
 - Expand dashboard customization options and filtering capabilities.
 - Improve scalability for larger numbers of honeypot deployments and concurrent attacks.
- Action items to implement in the next sprint:
 - Conduct final system testing and performance evaluation.
 - Complete project documentation and user guides or readmes for github.
 - Prepare the final project demonstration and presentation.

4.5 Sprint 9 : 7/8/2026 - 7/22/2026

4.5.1 Goal

Prepare SentinelGrid for final demonstration by completing testing, documentation, performance optimization, and presentation materials.

4.5.2 Sprint Planning

- What can we deliver by the end of this sprint?
 - Final dashboard
 - Complete documentation
 - Demo ready system
 - Final presentation
- How will we build it?

Focus on polishing existing features, fixing bugs, improving usability, and preparing demonstration materials.
- What must be done vs nice:
Must be done:
 - Final bug fixes
 - Documentation
 - User guide
 - Presentation
 - Demo preparation
Nice to have:
 - UI polish
 - Performance optimization
 - Additional analytics
 - Stretch features if time permits

4.5.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Completed end to end testing of the SentinelGrid platform, including the frontend, backend, machine learning pipeline, and Dockerized honeypot infrastructure
 - Fixed remaining bugs and improved system stability, performance, and overall usability
 - Finalized technical documentation, installation instructions, architecture diagrams, and user guides.
 - Prepared the final project presentation and demonstration showcasing adaptive honeypot deployment, behavioral analysis, and live attack monitoring
 - Performed performance evaluations to verify the system is within acceptable execution times
- What worked well:
 - Comprehensive documentation made deployment and system maintenance more straightforward.
 - Integration testing confirmed reliable communication between the dashboard, backend services, machine learning engine, and honeypot containers.

4.5.4 Sprint Retrospective

The team discussed:

- What went well during the sprint:
 - Successfully delivered all planned project objectives and integrated every major component into a cohesive platform
 - Final testing validated the stability and reliability of the complete SentinelGrid system
 - Documentation and user guides improved the accessibility and maintainability of the project